Part 3 [68 points]

1. **[34 points]** Describe and analyze your hand-crafted policy.

   (a) **[17 points]** With your scheduling policy, run the entire workflow **3 separate times**. For each run, measure the execution time of each batch job, as well as the latency outputs of memcached, running with a steady client load of 30K QPS. For each batch application, compute the mean and standard deviation of the execution time [1] across three runs. Also, compute the mean and standard deviation of the total time to complete all jobs - the makespan of all jobs. Fill in the table below. Finally, compute the SLO violation ratio for memcached for the three runs; the number of data points with 95th percentile latency $>$ 1ms, as a fraction of the total number of data points. The SLO violation ratio should be calculated during the time from when the first batch-job-container starts running to when the last batch-job-container stops running.

job name	mean time [s]	std [s]
barnes		
blackscholes		
canneal		
freqmine		
radix		
streamcluster		
vips		
total time		

   Answer:

   Create 3 bar plots (one for each run) of memcached p95 latency (y-axis) over time (x-axis), with annotations showing when each batch job started and ended, also indicating the machine each of them is running on. Using the augmented version of mcperf, you get two additional columns in the output: `ts_start` and `ts_end`. Use them to determine the width of each bar in the bar plot, while the height should represent the p95 latency. Align the x axis so that $x = 0$ coincides with the starting time of the first container. For each core in each machine show what job it is running using the colors proposed in this template (you can find them in `main.tex`). For example, draw a bar in the color of vips for core x from when vips is started to when it ends if vips ran on core x.

   Plots:

   (b) **[17 points]** Describe and justify the "optimal" scheduling policy you have designed.

   - Which node does memcached run on? Why?
     Answer:

   - Which node does each of the 7 batch jobs run on? Why?
     Answer:
     - barnes:

[1] Here, you should only consider the runtime, excluding time spans during which the container is paused.

  - blackscholes:
  - canneal:
  - freqmine:
  - radix:
  - streamcluster:
  - vips:

- Which jobs run concurrently / are colocated? Why?
  Answer:

- In which order did you run the 7 batch jobs? Why?
  Order (to be sorted): barnes, blackscholes, canneal, freqmine, radix, streamcluster, vips
  Why:

- How many threads have you used for each of the 7 batch jobs? Why?
  Answer:
  - barnes:
  - blackscholes:
  - canneal:
  - freqmine:
  - radix:
  - streamcluster:
  - vips:

- Which files did you modify or add and in what way? Which Kubernetes features did you use?
  Answer:

- Describe the design choices, ideas and trade-offs you took into account while creating your scheduler (if not already mentioned above). Describe how your policy (and its performance) compares to another policy that you experimented with (in particular to the second-best policy that you designed).
  Answer:

2. **[34 points]** Describe and analyze the AI-generated policy. [TODO: Xiaozhe]

   (a) **[17 points] Report the performance**:

   - Report the mean time of each batch job, as well as the makespan of all jobs for the AI-generated policy. Also, report the SLO violation ratio for memcached for the AI-generated policy. *Note: If the LLM failed to produce a valid solution after 3+ attempts, provide the error logs and your best-performing manual tweak for partial credit.*

job name	mean time [s] (Baseline)	mean time [s] (AI)
barnes		
blackscholes		
canneal		
freqmine		
radix		
streamcluster		
vips		
total time		

- Create 3 bar plots (one for each run), for the final AI-generated policy, of memcached p95 latency (y-axis) over time (x-axis), with annotations showing when each batch job started and ended, also indicating the machine each of them is running on. Use the same method as described in the previous question to create the bar plots.
  Answer:
- Create two line plots (one for the p95 latency and one for SLO violations) showing policy performance over iterations.
  Answer:

(b) **[17 points]** Analysis of the Evolutionary Process: Describe the process of how you used OpenEvolve to design the AI-generated policy. For example, you can include the following details in your answer (You are not limited to these questions. You can include any other details you find relevant and interesting). Your response will be graded based on the thoroughness of the exploration and clarity in explanation:

- How many iterations did you run through OpenEvolve with the LLMs to get to the final policy? Which LLM(s) did you use?
- How do you measure success? For example, how do you design the 'combined_score' or the success metric, given that there are two objectives (makespan and SLO violations)?
- Describe the initial policy you fed to the LLM(s) and why you chose this as the initial policy.
- Describe how the policy changes during the process, and how it impacted the performance of the policy (e.g., tail latency; SLO violations).
- Explain why did this AI-generated policy perform better (or worse) than the baseline?
- Identify one "hallucination" or logical flaw the model produced during the process and how you identified it.

Answer:

Please attach your modified/added YAML files, run scripts, OpenEvolve scripts, experiment outputs and the report as a zip file. You can find more detailed instructions about the submission in the project description file.

Important: The search space of all possible policies is exponential and you do not have enough credits to run all of them. We do not ask you to find the policy that minimizes the total running time, but rather to design a policy that has a reasonable running time, does not violate the SLO, and takes into account the characteristics of the first two parts of the project.